

BUILD WITH PHIL · SETUP GUIDE

Social Media Analytics From a Blank Mac to Live

A start-to-finish guide for someone who has never opened Terminal, never used GitHub, and never used Claude Code. Every step says what to type, what you should see, and what to do if you do not see it. Blue text is a link.

Live example

phillipkaraya.github.io/bwp-analytics-dashboard

Code

github.com/phillipkaraya/bwp-analytics-dashboard

Written

September 5, 2026

CONTENTS

1. Read this first
2. Words you will see
3. Part A · Set up the Mac
4. Part B · Get Claude Code, your assistant
5. Part C · Get the project and run it
6. Part D · Connect Chrome for scraping
7. Part E · Point it at the right accounts
8. Part F · The first data pull
9. Part G · Comment sentiment with Groq
10. Part H · Topics for the Content Vault
11. Part I · Publish it on GitHub Pages
12. The routine after setup
13. How the numbers are computed
14. When something goes wrong
15. File map and links

1. Read this first

You are about to set up a private analytics website for a creator's Instagram, TikTok, YouTube, Threads and LinkedIn. It shows followers, reach, engagement, every post, every comment with its sentiment, and a set of insights. It is hosted for free, protected by a PIN, and refreshed by a small program that reads the platforms through a Chrome browser that is signed in to those accounts. No platform API keys, no server, no monthly bill.

You do not need to know how to code. You need a Mac, about two hours, and the willingness to copy commands into Terminal and read what comes back. Along the way you will install [Claude Code](#), an assistant that runs in Terminal and can read this project, run these same commands for you, and explain any error in plain English. Everything in this guide can be done without it, but it turns most problems into a one-sentence question.

What you need before starting

A Mac running a recent macOS. Google Chrome. Login details for the creator's five social accounts. A free [GitHub](#) account. Optionally a free [Groq](#) account and a [Claude](#) account.

How long

About an hour of typing, plus two long waits that run on their own: the first data pull (15 to 30 minutes) and the first comment-labeling pass (about 25 minutes).

How commands work here

Dark boxes are things to type into Terminal, one line at a time, pressing Return after each. The grey line underneath says what you should see.

If you get stuck

Copy the error text and paste it into Claude Code with "this happened when I ran step X, what should I do?" It knows this project.

2. Words you will see

Terminal

The Mac app where you type commands. Open it with Spotlight: press Command and Space, type `Terminal`, press Return. Everything in the dark boxes goes here.

Repository (repo)

A folder of code with its full history, stored on [GitHub](#). "Cloning" copies it to your Mac. "Pushing" sends your changes back up.

Scraper

The Python program in the `scrape/` folder that reads each platform's pages and writes what it finds into JSON files the website reads.

Chrome DevTools Protocol (CDP)

A way for a program to drive a Chrome window that is already open. The scraper uses it so it can read the platforms as you, without ever handling your passwords. [Reference](#).

Static site and GitHub Pages

The website is plain files. [GitHub Pages](#) hosts plain files for free and republishes them every time you push.

PIN hash

The site never stores the PIN. It stores a scrambled fingerprint of it (a SHA-256 hash) and checks your entry against that.

Claude Code

Anthropic's command-line assistant. You open it inside the project folder and talk to it. It can read files, run commands with your permission, and fix things. [Documentation](#).

Groq

A company that runs open AI models very fast with a generous free tier. The project uses it once to label the sentiment of comments. [Console](#).

A Set up the Mac

Six installs, all free, all standard. If you already have any of them, the check command will say so and you can move on.

A1. Open Terminal and install Homebrew

[Homebrew](#) installs developer tools on a Mac. Paste the one line from its home page (it starts with `/bin/bash -c`) and follow the prompts; it asks for your Mac password once. When it finishes it prints two lines beginning "Next steps" that add Homebrew to your PATH. Run them. Then confirm:

```
brew --version
```

EXPECT A line like `Homebrew 4.x`.

A2. Git and the GitHub command line

```
brew install git gh
git --version
gh auth login
```

EXPECT `gh auth login` asks a few questions: choose *GitHub.com*, *HTTPS*, Yes to authenticate Git, and *Login with a web browser*. It shows a one-time code, opens the browser, you paste the code. Finish with `gh auth status`, which should name your GitHub account. [Reference](#).

A3. Node and pnpm

[Node](#) runs the website build. [pnpm](#) installs its packages. Node 20 or newer and pnpm 10 match what the publishing workflow uses.

```
brew install node
corepack enable
corepack prepare pnpm@10 --activate
node -v
pnpm -v
```

EXPECT `v20` or higher, and `10.x`. If `corepack` is not found, run `npm install -g pnpm@10` instead.

A4. Python and its one extra package

macOS ships Python 3. The scraper needs 3.9 or newer plus one library for the Chrome connection.

```
python3 --version
pip3 install websocket-client
```

EXPECT `Python 3.9` or higher. The first time you run `python3` macOS may offer to install command line developer tools; say yes and wait. If `pip3` complains about an "externally managed environment", run `pip3 install --user websocket-client` or `brew install python` and retry. [Package page](#).

A5. Google Chrome

Install [Chrome](#) if it is not already there, open it, and sign in to all five accounts: [Instagram](#), [TikTok](#), [YouTube](#), [Threads](#) and [LinkedIn](#). The scraper will read these pages through this Chrome, so it has to be signed in here, not in Safari or another browser.

CHECKPOINT A

`brew`, `git`, `gh`, `node`, `pnpm` and `python3` all answer with a version. `gh auth status` shows you are logged in. Chrome is signed in to all five platforms.

B Get Claude Code, your assistant

Claude Code is optional, and strongly recommended for a first-timer. It runs in Terminal, reads the project files, and can run any command in this guide for you while explaining what happened. You need a Claude account: a [Claude Pro](#) or [Max plan](#), or an [Anthropic Console](#) account with billing. The [quickstart](#) covers both.

```
npm install -g @anthropic-ai/claude-code
claude --version
```

EXPECT A version number. If you prefer an app with a window, the [Claude desktop app](#) includes the same assistant under its Code tab.

You use it by opening the project folder in Terminal (Part C shows the folder) and typing `claude`. The first run signs you in through the browser. Then you type in plain English. Useful things to say during this setup:

- "Read scrape/README.md and tell me what this project does."
- "Run `python3 scrape/run.py --platforms linkedin` and explain the output."
- "This error came up when I ran step C2. What does it mean and how do I fix it?" (paste the error)
- "Change the handles in scrape/handles.py to these accounts." (list them)
- "Commit the data folder and push it."

HOW PERMISSIONS WORK

Before Claude Code runs a command or edits a file it shows you exactly what it is about to do and asks. Say yes to things you understand, ask it to explain anything you do not. It will not ask you to paste passwords or keys into the chat; when a secret is needed it tells you which command to run yourself, and that is the correct behaviour.

C Get the project and run it

C1. Decide where the code lives, then clone it

Setting this up for the Build With Phil accounts? Clone Phil's repository, which already holds the data and handles:

```
mkdir -p ~/projects
cd ~/projects
git clone https://github.com/philipkaraya/bwp-analytics-dashboard.git
cd bwp-analytics-dashboard
pnpm install
```

Building your own dashboard for different accounts? Start from the [template repository](#) instead. It is the same code with empty data and placeholder handles. On that page click **Use this template**, then **Create a new repository**, give it a name (for example `social-analytics-dashboard`) and make it **Public** (GitHub Pages is free on public repositories). Then clone your new repository with the GitHub CLI, which fills in your username for you:

```
mkdir -p ~/projects
cd ~/projects
gh repo clone social-analytics-dashboard
cd social-analytics-dashboard
pnpm install
```

EXPECT A project folder under `~/projects` full of files, and `pnpm install` ending with a summary of packages added. From now on, every command in this guide assumes you are inside that folder. If you open a new Terminal window, `cd` into it first.

C2. Start it and open it

```
pnpm dev
```

EXPECT A few lines ending with a local address. Open <http://localhost:3000> in Chrome. You will see a PIN screen. Leave this Terminal window running; open a second window (Command N) for the next commands.

In development the PIN screen accepts a built-in fallback PIN so you can look around. Phil's repository already contains real data, so every tab shows numbers right away. A copy made from the template starts empty: the tabs open but show zeros until Part F. Either way, if the page loads, the code and the build tools are working.

C3. Choose your own PIN

The site stores only a hash of the PIN. Make one without the PIN appearing in your Terminal history (the first command waits silently for you to type the PIN and press Return):

```
read -s PIN; printf '%s' "$PIN" | shasum -a 256 | cut -d' ' -f1; unset PIN
```

EXPECT A 64-character string of letters and numbers. Copy it somewhere safe; you need it here and in Part I.

Create a file called `.env.local` in the project folder containing one line, with your hash in place of the placeholder. This file is ignored by git, so it never leaves your Mac:

```
NEXT_PUBLIC_DASHBOARD_PIN_HASH=paste-the-64-character-hash-here
```

Restart `pnpm dev` (press Control C in its window, run it again) and the PIN screen now accepts only your PIN.

CHECKPOINT C

The dashboard opens at localhost:3000, your PIN unlocks it, and all five tabs show data.

D Connect Chrome for scraping

The scraper never logs in to anything and never sees a password. It connects to a Chrome that is already running with a "remote debugging" door open on port 9222, opens its own tabs, reads the pages as you would see them, and closes only the tabs it opened. You open that door once per Chrome launch.

D1. Quit Chrome, then start it with the flag

```
osascript -e 'tell application "Google Chrome" to quit'
sleep 3
open -a "Google Chrome" --args --remote-debugging-port=9222
```

EXPECT Chrome reopens with your normal profile and your logins intact. Check the door is open:

```
curl -s http://127.0.0.1:9222/json/version
```

EXPECT A short block of text starting with `"Browser": "Chrome/..."`. If you get nothing, Chrome was not fully quit before relaunch; run the three lines again.

Chrome keeps the door open until you quit it. After a restart of the Mac, repeat D1. If Chrome is already running with the door open, the scraper just uses it; you never need to relaunch for each run.

TWO RULES

Do not close tabs the scraper opened while it is running, and do not run a second scrape at the same time. On Phil's own Mac a helper called `chrome-cdp` does D1 in one word and a tool called `ccbrowser` coordinates several automations sharing one Chrome; you do not need either. The scraper prints a harmless warning that `ccbrowser` was not found and carries on.

E Point it at the right accounts

All account handles live in one small file, `scrape/handles.py`. If you are running this for Build With Phil, it is already correct and you can skip to Part F. If you started from the template, the file holds placeholders; open it in any text editor (or ask Claude Code to change it) and replace the values. Phil's file looks like this:

```
HANGLES = {
    "instagram": "phillip.karaya",
    "tiktok": "phillip.karaya",
    "youtube": "@phillip.karaya",
    "threads": "phillip.karaya",
    "linkedin": "phillip-karaya",
}
YOUTUBE_CHANNEL_ID = "UCTb2yDMGoptvmgxh49UnIA"
```

- **Instagram, TikTok, Threads:** the username without the @. **YouTube:** the handle with the @. **LinkedIn:** the last part of the profile address, `linkedin.com/in/this-part`.
- **YouTube channel ID** is the one identifier that must be exact, because YouTube is read by channel ID so a handle rename can never break it. Find it in [YouTube Studio](#) under Settings, Channel, Advanced settings, or ask Claude Code: "find the channel ID for youtube.com/@handle".

F The first data pull

With Chrome's door open (Part D) and the accounts signed in, run one full sweep. It visits every profile to the end and writes one JSON file per platform into `public/data/`.

```
cd ~/projects/bwp-analytics-dashboard
python3 scrape/run.py --full
```

EXPECT 15 to 30 minutes. New tabs open and scroll in Chrome on their own; leave them alone. One line prints per platform as it finishes, for example `linkedin: +16 new, 16 total, followers 1714 (67s)`, then a final `done in ...: 5 ok, 0 failed` and a line starting `analyze:`. TikTok is the slow one.

Reload the dashboard at `localhost:3000` and the numbers now reflect what you just pulled. After this first sweep, the everyday command is the incremental run, which stops each platform once it has seen eight posts in a row that it already knows about, and takes a couple of minutes:

```
python3 scrape/run.py           # incremental, all five platforms
python3 scrape/run.py --platforms linkedin # one platform only
python3 scrape/run.py --analyze-only      # rebuild the derived files without scraping
```

FILE WRITTEN	WHAT IT HOLDS
<code>public/data/<platform>_posts.json</code>	every post: views, likes, comments, shares, date, caption, type
<code>public/data/scrape_state.json</code>	per-platform status, last run time, follower counts
<code>public/data/analytics.json</code>	every Insights section and the comment sentiment counts
<code>public/data/content_vault.json</code>	posts grouped by topic for the Content Vault
<code>public/data/follower_history.json</code>	one follower snapshot per day a scrape ran; appended, never rewritten

IF ONE PLATFORM FAILS

The others still finish. The message says which one and why; the usual cause is that Chrome is not signed in to it. Sign in, then rerun with `--platforms <name>`. LinkedIn in particular signs you out every few weeks by design and the scraper says so plainly.

G Comment sentiment with Groq

Every comment in `public/data/comments.json` gets a sentiment (positive, neutral or negative) and a separate "is this a question" flag. Two labelers exist. The **model** path sends comments in batches of 25 to [Groq](#), which runs open AI models on a free tier, and is the accurate one. The **lexicon** path is a built-in word and emoji matcher that works with no account at all and fills in whenever the model is unavailable. You can skip this part entirely and the dashboard still works; the labels will just be less accurate.

G1. Get a free key and store it in the Mac Keychain

Sign up at [console.groq.com](#), open [API Keys](#), create one, and copy it. Store it in the Keychain so it never sits in a file. The command prompts for the key and hides what you type:

```
security add-generic-password -a "$USER" -s GROQ_API_KEY -w
```

EXPECT It asks for the password twice (that is the key) and returns silently. You will not need to paste the key again.

G2. Run the labeling pass

This reads the key from Keychain, hands it to the analyzer for the length of one command, and labels every comment that has no current label:

```
GROQ_API_KEY="$(security find-generic-password -a "$USER" -s GROQ_API_KEY -w)" python3  
scrape/run.py --analyze-only
```

EXPECT The first pass over 8,000 comments makes about 320 requests and takes 20 to 30 minutes. Every ten requests a progress line prints, like `sentiment: 725 labeled by model so far, 30 calls, 1.7s per call, errors {'http429:gpt-oss-120b': 9}`. Those 429 counts are normal: Groq's free models allow about 8,000 tokens a minute, so a busy model is told to wait and the run moves on to the next one. The final line reads `sentiment: N by model, M by lexicon, ...`. If M is not zero, run the same command a second time; it only sends the leftover rows and takes a few minutes.

WHAT GOOD LOOKS LIKE

On the Build With Phil corpus the labels came out positive 5,300 (66%), neutral 2,187 (27%), negative 512 (6%), with 559 questions, all labeled by the model. Reload the Comments tab and the Sentiment panel shows the same split.

ON PHIL'S MAC

The same thing runs through a personal Keychain helper: `secret-sync run GROQ_API_KEY -- python3 scrape/run.py --analyze-only`. The two commands are equivalent; use whichever is on the machine in front of you. Rate limits are documented on [Groq's rate limits page](#).

H Topics for the Content Vault

The Content Vault files every post under topics such as Comedy, Cars or Finance. The rules are a plain text file, `scrape/categories.json`: each topic has a label and a list of keywords, matched as whole words against the post's title, caption and hashtags. A post can land in several topics; one that matches nothing is filed under Other. To tune it, edit the keywords (or tell Claude Code "add a topic called Travel with these keywords"), then rebuild without scraping:

```
python3 scrape/run.py --analyze-only
```

EXPECT Under a second. The Vault's header shows how many posts are still unfiled, so you can see whether a new keyword helped.

I Publish it on GitHub Pages

The repository includes a GitHub Actions workflow, `.github/workflows/deploy.yml`. Every time you push to the `main` branch it builds the site and publishes it to GitHub Pages. Three one-time settings make that work.

I1. Turn on Pages

Open your repository on GitHub, click **Settings**, then **Pages** in the left column, and under **Build and deployment** set **Source** to **GitHub Actions**. [GitHub's instructions](#).

I2. Give the workflow your PIN hash

The live site needs the same hash you made in C3. It goes into a repository secret named `DASHBOARD_PIN_HASH`. The first command waits silently for you to paste the hash:

```
read -s HASH; gh secret set DASHBOARD_PIN_HASH --body "$HASH"; unset HASH
```

EXPECT ✓ Set Actions secret DASHBOARD_PIN_HASH. [About secrets](#).

I3. The address takes care of itself

Pages serves a project at `https://<your-username>.github.io/<repository-name>/`. The workflow reads the repository name at build time and bakes that path in, so whatever you named your repository, nothing needs editing here.

I4. Push and watch

```
git add public/data
git commit -m "First data pull"
git push
gh run watch
```

EXPECT `gh run watch` shows the build and deploy jobs turning green over about a minute. Then open `https://<your-username>.github.io/<repository-name>/`, enter your PIN, and you are looking at the live site. If `git commit` asks who you are, run the two `git config --global` lines it suggests with your name and email, then commit again.

WHY THE SCRAPER STAYS ON YOUR MAC

The site itself is just files, so GitHub can host it. The scraper needs your signed-in Chrome, which only exists on your Mac. So data flows: scrape on the Mac, commit, push, GitHub rebuilds. That is the whole architecture.

The routine after setup

Whenever new posts have gone out, or once a day if you want a daily follower point:

```
cd ~/projects/bwp-analytics-dashboard
python3 scrape/run.py
git add public/data && git commit -m "Refresh data" && git push
```

Three lines, a few minutes. Some habits that keep it healthy:

- **Chrome's door has to be open** (Part D). After a Mac restart, run D1 again before scraping.
- **LinkedIn once a day at most.** It drives your real session with human-like pauses. When it reports a login wall, sign in to linkedin.com in Chrome and rerun with `--platforms linkedin`.
- **A full sweep now and then, not on a schedule.** `--full` refreshes the counts on every old post and takes up to half an hour.
- **Re-run the Groq command only when comments change.** Labels are cached; a rerun costs a handful of requests.
- **Follower history grows one point per day a scrape ran.** Rebuilding on old data never adds a point.

How the numbers are computed

NUMBER	RULE
Overview window	Your numbers for the last 7, 30, 60 or 90 days, picked with the switch at the top. Until you pick one, the shortest of 30, 60 or 90 days that contains a post. The change shown compares against the window before it.
Top Performing Posts	Its own 30, 60 or 90 day switch and a chip per platform. Uncheck a platform to leave it out; the table re-ranks at once. It starts on the window the Overview is showing.
Reach	Views where the platform publishes them. Threads and LinkedIn publish no view count, so their reach is likes and the charts label the bar "(likes)". Likes are never added into a views total.
Engagement rate	Likes plus comments plus shares, divided by views. For Threads and LinkedIn, divided by followers instead.
Averages in Insights	Only posts with a view count above zero are averaged, so the viewless platforms never drag averages down.
Follower change	From the last two points in the follower history. A platform added later starts its line at its first point.
Comment sentiment	Labels from the model, lexicon as fallback, stored per comment. The website never re-classifies; a missing label counts as neutral.
Questions	A separate flag, independent of sentiment, ranked by likes.

When something goes wrong

YOU SEE	WHAT IT MEANS	WHAT TO DO
<code>command not found</code> after any install	Terminal has not picked up the new tool yet.	Close the Terminal window and open a new one. For Homebrew, run the two "Next steps" lines it printed.
<code>curl</code> to port 9222 prints nothing	Chrome is not running with the debugging door open.	Repeat Part D1: quit Chrome completely, wait, relaunch with the flag.
A platform says "login" or "checkpoint wall"	Chrome is not signed in to it, or the session expired (LinkedIn does this by design).	Sign in to that site in Chrome, then <code>python3 scrape/run.py --platforms <name></code> .

Instagram returns 429 or an HTML page	Instagram throttles scripted requests.	Already handled by reading the page's own responses. If it persists, wait ten minutes and rerun.
TikTok "account not found" or YouTube 404	The handle in <code>scrape/handles.py</code> is wrong or was renamed.	Fix the handle (Part E) and rerun.
Groq 403 with error code 1010	A Cloudflare user-agent block, not a bad key.	The scraper already sends a browser user agent; rerun. If you built your own script, add one.
Groq log full of 429 counts	Normal rate limiting on the free tier.	Let it finish. Run the command a second time if the final line shows rows still "by lexicon".
"REFUSED: memory critical"	The Mac is too low on memory to scrape safely.	Close heavy apps and tabs, then rerun.
Dev site shows old styles after an edit	The build tool's cache.	Stop <code>pnpm dev</code> , run <code>rm -rf .next/dev .next/cache</code> , start it again.
Live site still shows old data after a push	The publish job has not finished, or the browser cached the files.	<code>gh run list --limit 1</code> to check; then reload with Shift held.
Anything else		Open Claude Code in the project folder and paste the exact error with the step number. That is what it is for.

File map and links

```

app/                one page per tab
components/         the pieces of each page (charts, layout, overview, posts, comments, insights, vault)
lib/                shared logic: data loading, calculations, formatting, the PIN check
scrape/
  run.py            the command you run: scrape, then analyze
  analyze.py        builds analytics.json, content_vault.json, follower_history.json, labels comments
  sentiment.py      comment labeling: Groq models with rate-limit handling, lexicon fallback
  categories.json   topic keywords (edit freely)
  handles.py        account handles and the YouTube channel ID
  cdp.py            the Chrome connection
  platforms/        one reader per platform: instagram, tiktok, youtube, threads, linkedin
public/data/         the JSON files the site reads; commit these after every scrape
.github/workflows/deploy.yml builds and publishes on every push to main
docs/               this guide and its source

```

LINK	USE
Live dashboard	The finished example this guide builds.
Phil's repository	Build With Phil's instance: code, data, this guide.
Template repository	Click Use this template to start your own copy with empty data.
Claude Code docs	Install, sign in, how permissions and prompts work.
Claude desktop app	Same assistant with a window instead of Terminal.
Homebrew	Installs git, gh, node and python on a Mac.

GitHub CLI	<code>gh auth login</code> , <code>gh secret set</code> , <code>gh run watch</code> .
pnpm	Installs and runs the website's packages.
GitHub Pages docs	Hosting, custom domains, troubleshooting a deploy.
Groq API keys	Create the free key used for comment sentiment.
Groq rate limits	Why 429s appear and what the free tier allows.
Chrome DevTools Protocol	What port 9222 is and how the scraper talks to Chrome.
Next.js docs	The framework the site is built with, if you want to change the pages.

Rules that keep this safe

- **No secrets in the repository, ever.** The PIN is a hash in a GitHub secret. The Groq key is in the Keychain. Before committing, `git status` should never list `.env.local` or anything with a key in it.
- **The scraper uses your Chrome, it does not copy it.** Cookies and login data are never copied out. It opens its own tabs and closes only those.
- **LinkedIn messaging is off limits.** The scraper reads the public activity feed and the follower count on the profile. Nothing from messages is read, stored or logged.
- **Measure what a platform publishes.** Threads and LinkedIn have no view counts, so the dashboard shows likes as reach rather than zeros.
- **Ask before you guess.** Claude Code can read every file in this project. When a step does not match what you see, ask it why before changing anything.